

Theory of Automata

CS-3005

Lecture 1

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Contents

- Outline
- Objectives
- Background
- Motivation & Description

Course Outlines

Finite State Models

- Language definitions preliminaries
- Regular expressions/Regular languages
- Finite automata (FAs)
- Transition graphs (TGs)
- NFAs, kleene's theorem
- *Making a Scanner*
- Transducers (automata with output)
- Pumping lemma and non regular language

Grammars and PDA

- Context free grammars
- Derivations, derivation trees and ambiguity
- Operations preserving CFLs
- Normal form grammars and parsing

Push Down Automata

- *Making a Parser*
- Pumping lemma and non-context free languages
- Decidability
- Chomsky's hierarchy of grammars

Turning Machines Theory

- Turing machines
- Post machine
- *Turing Machine project*
- Variations on TM
- TM encoding, Universal Turing Machine
- Context sensitive Grammars
- Defining Computers by TMs

What is Automata Theory?

- Study of **abstract computing devices, or “machines”**
- Automaton = an abstract computing device
 - Note: A “device” need not even be a physical hardware!
- A fundamental question in computer science:
 - Find out what **different models of machines can do and cannot do**
 - The theory of computation
- Computability vs. Complexity

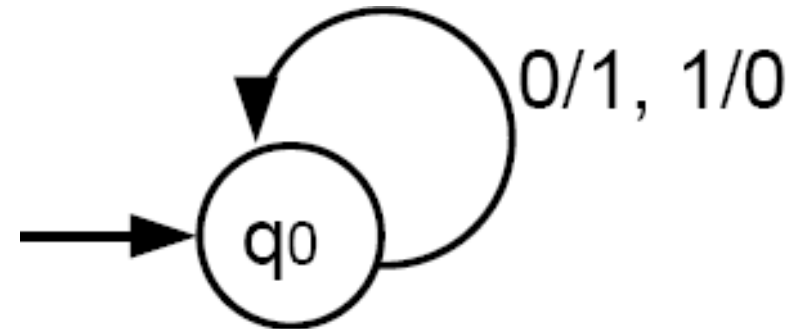
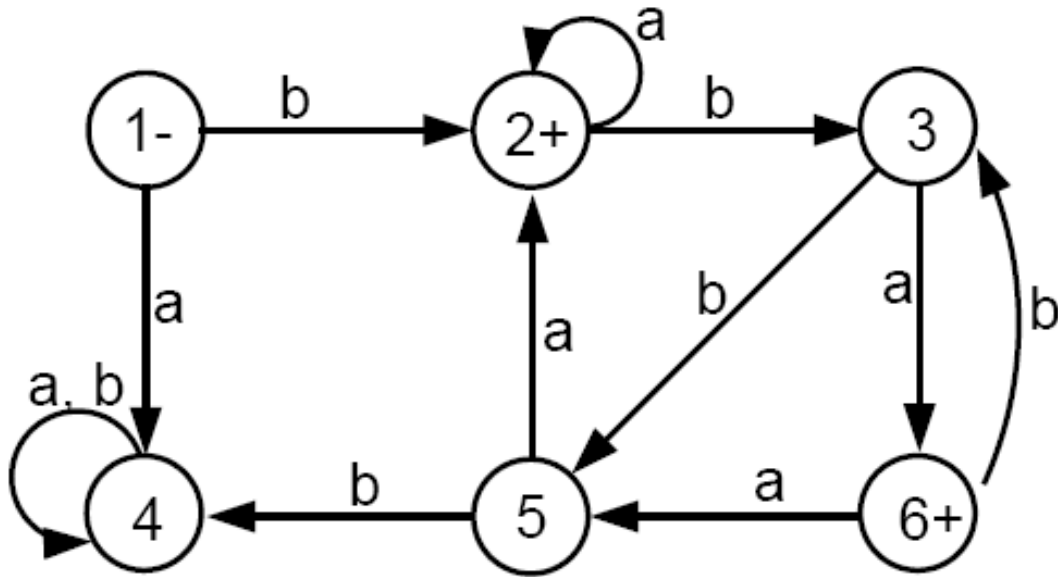
Study of Languages language vs computational model

- In independent work in Mathematical Logic, it was noted that
 - There is a natural correspondence between study of models of Computation and
 - The study of natural language Linguistics in an **abstract and mathematical** sense.
 - **Noam Chomsky** created the subject of mathematical model for description of languages
 - His study grew to the point **where it began to shed light on the study of computer languages**

The Theory of Formal Languages

- Regular Languages RE / FSM
- Context free languages CFG / PDA
- Context Sensitive Languages L Bounded TM
- Recursively enumerable Languages TM
- Alan Turing's theoretical model for an algorithm machine employing very **simple set of mathematical structure held out the possibility** that a physical model of the turnings idea could be actually build.
- 1st computer was built and Turing was part of it in Manchester University.

Sample Work



Practical Applications

- FA
 - Text processing, compilers and hardware design
- CFGs
 - Programming Languages and AI
- Cellular Automata
 - Biology
- Digital physics
 - Whole universe is computed by some sort of a discrete automation

Languages & Grammars

- Languages: “A language is a **collection of sentences** of finite length all constructed from a finite alphabet of symbols”
- Grammars: “A grammar can be regarded as a device that **enumerates the sentences of a language**” - nothing more, nothing less

Languages

- In English, we distinguish 3 different entities
 1. Letter
 2. Words
 3. Sentences
- Group of letter make up words and groups of words make up a sentence.
 - However, not all collection of letter form valid words, and not all collections of words form valid sentences
- This situation also exists in computer langauges.

Languages

- To construct a general theory of formal languages,
 - we need to have a definition of a language structure, in which the
 - decision of whether a given string of units constitutes a valid larger unit is not a matter of guesswork, but is based on explicitly stated rules.
- In this model, language will be considered as symbols with formal rules, and **not as expressions of ideas in the minds of humans.**
- The term “formal” **emphasizes that it is the form of the string** of symbols that we are interested in, not the meaning.

Basic Definition

- A finite non-empty set of symbols(letters) is called an alphabet. It is denoted by Σ *greek Letter called Sigma*.
- Example:
- $\Sigma = \{a, b\}$
- $\Sigma = \{0, 1\}$

String

- Concatenation of finite symbols from the alphabet is called a string.
- Example:
 - If $\Sigma = \{a, b\}$, then
 - a, abab, aaabb, ababababababb,

Words

- Words are strings belongs to some language.
- Example:
 - if $\Sigma = \{x\}$, then a language L can be defined as
$$L = \{ x^n : n = 0, 1, 2, 3, \dots \} \quad \text{or } L = \{ \Delta, x, xx, xxx, \dots \dots \dots \}$$
 - Here x, xx, are the words of L.

All words are strings but not all strings are words.

Practice:

- What Is The Difference Between The Strings And The Words Of A Language?

.

Practice:

- What Is The Difference Between **The Strings And The Words** Of A Language?

A string is any combination of the letters of an alphabet where as the words of a language are the strings that are always made according to certain rules used to define that language.

Practice:

- What Is The Difference Between **The Strings And The Words** Of A Language?

Example:

Alphabet $\Sigma = \{ a , b \}$ Here a , b are the letters of this alphabet.

As you can see we can make a lot of strings from these **letters a and b**.

For example **a,b,aa,ab,ba,bb,aaa,aab,aba,baa**

Practice:

- What Is The Difference Between **The Strings And The Words** Of A Language?

Example:

we define a language over **this alphabet having no a's and only odd number of b's**. Then the words of this language would have only those strings that have only odd number of b's and no a's. some example words of our defined language are **b , bbb , bbbbbb , bbbbbbbb , and so on.**

EMPTY STRING or NULL STRING

- We shall allow a string to have no letters. We call this empty string or null string, and denote it by the symbol Λ .
- For all languages, the null word, if it is a word in the language, is the word that has no letters. We also denote the null word by Λ .
- Two words are considered the same if all their letters are the same and in the same order.
- For clarity, we usually do not allow the symbol Λ to be part of the alphabet of any language.

Discussion of NULL

- The language that has no words is denoted by the standard symbol for null set, $\emptyset$.
- It is not true that Λ is a word in the language $\emptyset$ since this language has no words at all.
- If a certain language L does not contain the word Λ and we wish to add it to L , we use the operation “+” to form $L + \{\Lambda\}$. This language is not the same as L .
- However, the language $L + \emptyset$ is the same as L since no new words have been added.

Introduction to Defining Languages

- The rules for defining a language can be of two kinds:
 1. Testing
They can tell us how to test if a string of alphabet letters is a valid word, or
 2. Procedure
They can tell us how to construct all the words in the language by some clear procedures.

Defining Language

- Example: Consider this alphabet with only one letter

$$\Sigma = \{ x \}$$

- We can define a language by saying that any nonempty string of alphabet letters is a word

$$L1 = \{ x, xx, xxx, xxxx, \dots \} \text{ or}$$

$$L1 = \{ x^n \text{ for } n = 1, 2, 3, \dots \}$$

- Note that because of the way we have defined it, the language L1 does not include the null word Λ .

Example

- The language L of strings of odd length, defined over $\Sigma=\{a\}$, can be written as

$$L=\{a, aaa, aaaaa, \dots\}$$

- Example:

The language L of strings that does not start with a , defined over $\Sigma=\{a,b,c\}$, can be written as

$$L=\{b, c, ba, bb, bc, ca, cb, cc, \dots\}$$

Exmample:

The language L of strings of length 2, defined over $\Sigma=\{0,1,2\}$, can be written as

$$L=\{00, 01, 02,10, 11,12,20,21,22\}$$

- Example:

The language L of strings ending in 0, defined over $\Sigma =\{0,1\}$, can be written as

$$L=\{0,00,10,000,010,100,110,\dots\}$$

Concatenation

- Let us define an operation, concatenation, in which two strings are written down side by side to form a new longer string.

xxx concatenated with xx is the word xxxxx

x^n concatenated with x^m is the word x^n+x^m

- For convenience, we may label a word in a given language by a new symbol. For example,

xxx is called a, and xx is called b

Concatenation

- Then to denote the word formed by concatenating a and b, we can write

$$ab = \text{xxxxxx}$$

- It is not true that when two words are concatenated, they produce another word. For example, if the language is

$$L2 = \{x, xxx, xxxxx, \dots\} = \{x^{2n+1} \text{ for } n = 0, 1, 2, \dots\}$$

then $a = xxx$ and $b = xxxxx$ are both words in $L2$, but their concatenation $ab = \text{xxxxxxxx}$ is not in $L2$

Concatenation makes new Words?

- Note that in this simple example, we have:

$$ab = ba$$

But in general, this relationship does NOT hold for all languages (e.g., houseboat and boathouse are two different words in English).

- Example: Consider another language by beginning with the alphabet

$$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$$

Define the language

$$L3 = \{ \text{any finite string of alphabet letters that does not start with the letter zero} \}$$

Example

- This language L_3 looks like the set of positive integers:

$$L_3 = \{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, \dots \}$$

- If we want to define L_3 so that it includes the string (word) 0, we could say

$$L_3 = \{ \text{any finite string of alphabet letters that, if it starts with a 0, has no more letters after the first} \}$$

Length

- We define the function length of a string to be the number of letters in the string.
- Example:
 - If $a = \text{xxxx}$ in the language $L1$, then $\text{length}(a) = 4$
 - If $c = 428$ in the language $L3$, then $\text{length}(c) = 3$
 - If $d = 0$ in the language $L3$, then $\text{length}(d) = 1$
 - In any language that includes the null word Λ , then $\text{length}(\Lambda) = 0$
 - For any word w in any language, if $\text{length}(w) = 0$ then $w = \Lambda$.

Example

- Recall that the language $L1$ does not contain the null string Λ . Let us define a language like $L1$ but that does contain Λ :

$$\begin{aligned} L4 &= \{ \Lambda, x, xx, xxx, xxxx, \dots \} \\ &= \{ x^n \text{ for } n = 0, 1, 2, 3, \dots \} \end{aligned}$$

- Here we have defined that

$$x^0 = \Lambda \text{ (NOT } x^0 = 1 \text{ as in algebra)}$$

- In this way, x^n always means the string of n alphabet letters x 's.
- Remember that even Λ is a word in the language, it is not a letter in the alphabet.

Reverse

- If a is a word in some language L , then $\text{reverse}(a)$ is the same string of letters spelled backward, even if this backward string is not a word in L .
- Example:
 - $\text{reverse}(xxx) = xxx$
 - $\text{reverse}(145) = 541$
 - Note that 140 is a word in L_3 , but $\text{reverse}(140) = 041$ is NOT a word in L_3

This language L_3 looks like the set of positive integers:

$$\underline{L_3 = \{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, \dots \}}$$

If we want to define L_3 so that it includes the string (word) 0, we could say

$$\underline{L_3 = \{ \text{any finite string of alphabet letters that, if it starts with a 0, has no more letters after the first} \}}$$

Palindrome

- Let us define a new language called Palindrome over the alphabet

$$\underline{\Sigma = \{ a, b \}}$$

- $\text{PALINDROME} = \{ \Lambda, \text{ and all strings } x \text{ such that } \text{reverse}(x) = x \}$
- If we want to list the elements in PALINDROME, we find

$$\underline{\text{PALINDROME} = \{ \Lambda, a, b, aa, bb, aaa, aba, bab, bbb, aaaa, abba, \dots \}}$$

Palindrome

- Sometimes two words in PALINDROME when concatenated will produce a word in PALINDROME
- abba concatenated with abbaabba gives abbaabbaabba (in PALINDROME)
- But more often, the concatenation is not a word in PALINDROME
- aa concatenated with aba gives aaaba (NOT in PALINDROME)
- The language PALINDROME has interesting properties that we shall examine later.

Practice Questions

1. Consider the language S^* , where $S = \{a, b\}$. How many words does this language have of length 2? of length 3? Of length n ?
2. Consider the language S^* , where $S = \{aa, b\}$. How many words does this language have of length 4? of length 5? Of length 6? What can be said in general?
3. Consider the language S^* , where $S = \{ab, ba\}$. Write out all the words in S^* that have seven or fewer letters. Can any word in this language contain the substrings aaa or bbb ?